



МОСКОВСКИЙ УНИВЕРСИТЕТ ИМЕНИ С.Ю. ВИТТЕ

Факультет Информационных технологий

Кафедра Кафедра информационных систем

Направление подготовки/Специальность Прикладная информатика

ОТЧЕТ

о прохождении

Производственная практика

/вид практики/

Эксплуатационная практика

/тип практики/

Студента Спиридонова Никиты Алексеевича
(фамилия, имя, отчество)

Место прохождения практики ЧОУВО «МУ им. С.Ю. Витте»

Период прохождения практики с 20.04.2026 г. по 31.05.2026 г.

г. Москва 2026 г.

Оглавление

1. Введение	3
2. Цель и задачи	3
3. Теоретическая часть	4
4. Установка и настройка среды	7
5. Выполнение практической работы	8
5.1 Первый запуск YOLOv8	8
5.2 Анализ выходных данных модели	10
5.3 Обработка видео	10
5.4 Фильтрация объектов по классам	11
5.5 Подсчет объектов в реальном времени	13
5.6 Реализация зоны интереса (ROI)	14
5.7 Исследование порога уверенности	15
5.8 Изучение метрик качества	17
6. Вывод	18
Список используемой литературы	20

1. Введение

Компьютерное зрение (или же CV) - это стремительно развивающаяся область искусственного интеллекта, позволяющая наделить компьютеры способностью интерпретировать визуальную информацию из изображений и видео. Оно дает машинам способность анализировать изображения и видео, извлекая из них информацию. Одни системы стремятся воспроизвести принципы человеческого зрительного восприятия, другие опираются на анализ больших объемов данных и статистические модели, а третьи используют геометрические принципы для решения поставленных задач.[1] В нынешних реалиях компьютерное зрение активно применяется в различных сферах жизни, от видеонаблюдения и медицины до самоуправляемых автомобилей и робототехники.

Ключевым элементом CV является обнаружение объектов. Это означает, что система может автоматически находить объекты на картинке или видео, определять их местоположение и классифицировать (то есть разделять на классы, например "кот" или "собака"). В определении местоположения важно точно указывать границы объектов (для этого используются bounding box или же как ее часто еще называют b-box).

В рамках данной практической работы мною будет рассматриваться использование модели YOLOv8 для детектирования объектов на изображении, видеофайлах а также в режиме реального времени с использованием веб-камеры. Также перед о мной стоит цель изучить основные параметры работы модели, методы фильтрации результатов, использование зоны интереса (или же иначе ROI) для подсчета объектов и базовые метрики оценки качества детектирования.

2. Цель и задачи

Данная практическая работа направлена на освоение современных методов обнаружения объектов с помощью нейросетей, в частности, на

примере YOLOv8. Будет изучено как работают эти системы и научимся применять готовую модель для поиска, анализа и подсчета объектов на изображениях, видео и в потоковом режиме. В ходе работы мы установим и настроим нужное ПО, протестируем модель на разных данных, разберемся с ее выходных данных и научимся обрабатывать полученные результаты. Также мною будет изучено как нужно правильно настраивать параметры для улучшения качества обнаружения и понимания основных показателей эффективности моделей компьютерного зрения.

3. Теоретическая часть

Нейронная сеть представляет собой вычислительную модель, вдохновленную структурой и функциями биологических нейронных сетей(например нейроны головного мозга). Обычно, нейроны объединены в слои, разные слои могут выполнять разные преобразования своих входных сигналов. Сигналы передаются от входного слоя к выходному слою, иногда проходя через несколько промежуточных (скрытых) слоев. Глубокой нейронной сетью обычно считают если она имеет минимум два скрытых слоя. [2] В отличие от традиционного подхода к программированию, в нейронной сети мы не говорим компьютеру, как решить нашу проблему. Вместо этого он учится на основе наблюдательных данных, самостоятельно находя решение поставленной задачи. [3]

Нейроны - это узлы в нейросети, которые служат точками соединения. Благодаря взаимодействию входных данных с весами и смещениями, эти узлы приобретают способность к распознаванию, классификации и детальному описанию объектов, содержащихся в информации. Они функционируют благодаря нескольким ключевым механизмам. Прямое распространение - процесс, при котором информация последовательно проходит через слои сети, начиная с входных данных и заканчивая выходным результатом, что помогает сети делать прогнозы или классифицировать

информацию. Обратное распространение использует алгоритм для выявления неточностей прогнозирования и корректирует веса смещения, двигаясь назад по слоям глубокой обучающей сети для обучения модели. А третьим механизмом является функция активации, которая влияет на принятие решений сетью, облегчает обучение на данных и конечно улучшает сходимость нейронной сети. [4]

YOLO - это продвинутый алгоритм обнаружения объектов, использующий глубокие нейронные сети. Она расшифровывается как "You Only Look Once" или же если переводить "Вы смотрите только один раз". Данная аббревиатура прямо описывает принцип работы модели, фото которые приходят на вход нейронной сети анализируются один раз и сразу же предоставляют результат обнаружения объекта.

Главной целью обнаружения объекта является не только проверка на наличии объекта на изображении, а также еще определение его точного местоположения. Для этого модель определяет координаты ограничивающей рамки вокруг объекта и присваивает ему определенный класс. На фото может быть сразу несколько объектов разных классов, а также они могут пересекаться.

В отличие от остальных подходов, YOLO превосходит их благодаря сверточным нейронным сетям. Модель может анализировать все изображение целиком, учитывать контекст других объектов и выполнять вычисления чрезвычайно быстро, что очень часто используют в анализе данных в реальном времени.

Архитектура YOLOv8 разделена на три функциональных модуля. Первый модуль Backbone ответственен за извлечение признаков из входного изображения при помощи C2f-блоков. Neck объединяет признаки на разных уровнях с использованием FPN и PAN. И финальный модуль Detection Head предсказывает местоположение, класс, и уверенность объектов. Модель

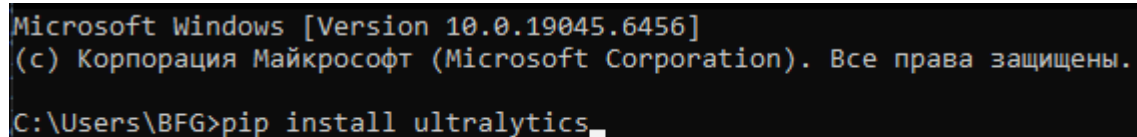
YOLOv8n содержит 225 слоев, в то же время у YOLOv8x, которая является самой большой из семьи v8, 365 слоев. Функцией активации является SiLU(функция сигмоиды).

В контексте обработки изображений свертка включает в себя скользящее небольшое филътра, которое очень часто называют ядром, по входному изображению для создания карты значимых признаков. Она позволяет моделям искусственного интеллекта автоматически обучаться и находить закономерности, такие как края, текстуры и формы, без вмешательства человека. При помощи свертки компьютерное зрение может создавать иерархическое понимание визуальных данных, начиная с простых линий и переходя к сложным объектам, таким как лица или транспортные средства.

Для оценки классификации изображений используются такие метрики, как Accuracy, Precision, Recall, F1-score, и матрица ошибок. Для моделей обнаружения объектов наиболее эффективными являются IoU и mAP. IoU или же "Intersection over Union" показывает насколько предсказанный прямоугольник совпал с реальным расположением объекта. Вычисляется он как отношение площади пересечения двух прямоугольников к площади их объединения. Значения данной метрики изменяются от 0 до 1. Если значение превышает 0,5 обычно детекцию считают удачной. Также есть такая метрика Confidence, она отражает уверенность модели в том, что обнаруженный объект принадлежит определенному классу. Это значение также может изменяться от 0 до 1. Чтобы понять, насколько хорошо работает детектор, мы смотрим на Precision и Recall. Первая метрика определяет процент истинно положительных срабатываний от общего числа обнаружений, сделанных детектором. А вторая показывает, какая часть из всех настоящих объектов детектор смог найти.

4. Установка и настройка среды

Для корректного выполнения практики нужно правильно настроить программную среду, в которой будет выполняться работа. В рамках выполнения этой работы был выбран язык программирования Python. В дополнении к нему нам необходимо установить набор специализированных библиотек для задач компьютерного зрения и машинного обучения. Компания Ultralytics является ведущим разработчиком в области компьютерного зрения, а также известна как разработчик моделей детекции семейства YOLO. Данная компания распространяет YOLO через Python-пакет ultralytics, который нужно ставить через пакетный менеджер pip.



```
Microsoft Windows [Version 10.0.19045.6456]
(c) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

C:\Users\BFG>pip install ultralytics_
```

Рисунок 1 - Установка ultralytics

У Python есть несколько реализаций интерпретатора. Они отличаются языками программирования, на котором они написаны. Например, PyPy - компилятор написанный на Python. Также есть кросс-компилятор Jython, он написан на Java и при выполнении компилирует исходный код Python в байт-код Java. Но я выбрал самую популярную реализацию CPython, то есть написанную на языке C. У него полная поддержка языка Python и много библиотек переписаны на низкоуровневые языки(что в свою очередь добавляет в скорости выполнения).

Библиотека Ultralytics YOLO для Python обеспечивает бесшовную интеграцию в проекты на Python, упрощая загрузку, запуск и обработку результатов работы модели. Он минимизирует сложность работы с моделями, позволяя легко загружать их, запускать и обрабатывать результаты. Также при помощи него облегчается внедрение функции детекции в существующие и новые проекты. У нее обширная документация и большая база пользователей.

OpenCV или же "Open Source Computer Vision Library" является одной из самых распространенных библиотек для разработки приложений и исследований в области компьютерного зрения. Она очень сильно упрощает работу с изображениями, тем что предоставляет готовые инструменты для их захвата, обработки и анализа. Благодаря своей универсальности, OpenCV активно применяется как в науке, так и в бизнесе. Он также использует удобный формат данных, совместимый с популярными научными библиотеками, такими как NumPy и SciPy. Что позволяет комбинировать их и ускорять разработку.[5]

5. Выполнение практической работы

5.1 Первый запуск YOLOv8

Была выполнена установка всех программ и зависимостей. Для решения поставленных задач была выбрана модель YOLOv8n (Nano), которая является самой быстрой и наименьшей по размеру моделей из всего семейства. Эта модель обучена 80 классам из датасета COCO. Он является одним из самых известных открытых датасетов в мире. В его базе лежит около 330 тысяч фото.

Для выполнения первой подзадачи нужно импортировать из модуль ultralytics класс YOLO, на базе этого класса строится вся работа с YOLOv8. Чтобы выбрать модель нужно передать в класс YOLO название модели, в нашем случае yolov8n.pt, и после запуска библиотека сама ее загрузит в папку с скриптом. После чего на вход нужно подать само изображение указав путь к нему и при помощи функции show() можно сразу же вывести результат. Библиотека откроет изображение с отмеченными ограничивающими прямоугольниками (b-box) с метками класса и степенью уверенности, с которой модель его распознала.

При первом запуске я разобрался как работать с библиотекой и проверил корректность ее работы.



Рисунок 2 – Входное изображение

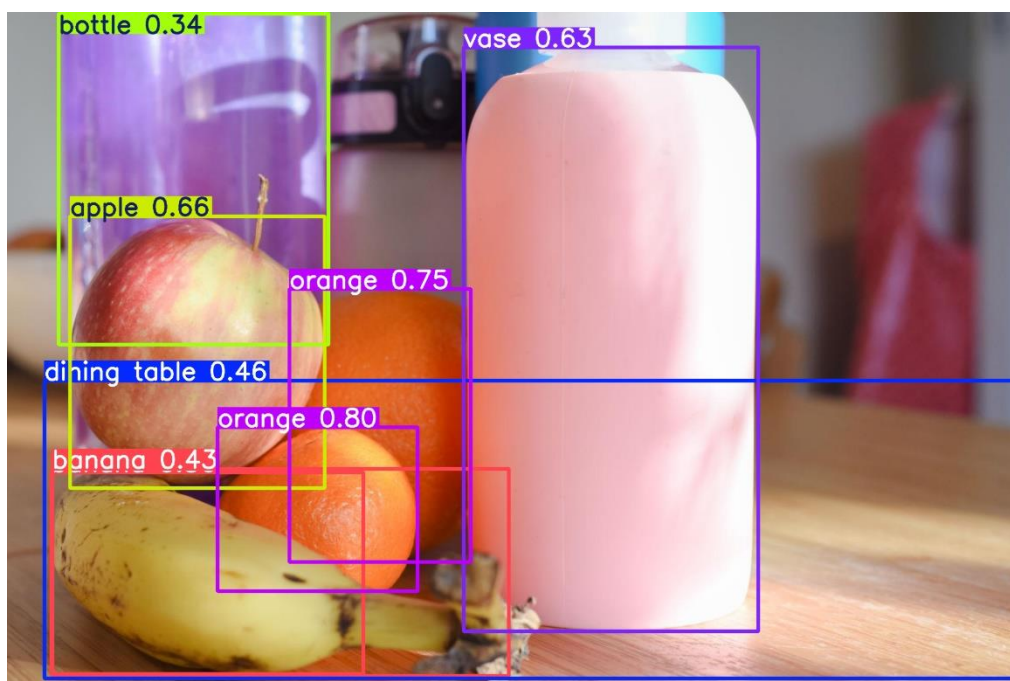


Рисунок 3 - Выходное изображение

На рисунках 2 и 3 приведены результаты работы модели без дополнительных настроек. Как мы видим модель корректно определяет координаты объектов на фото.

5.2 Анализ выходных данных модели

Следующим этапом было изучить какие данные возвращает модель и как с ними можно работать. В результате модель вернула объекты с типом Results, в котором были вся детальная информация о найденных объектах. В данной работе нам могут понадобиться координаты детектированных объектов, их идентификаторы, и также значения уверенности модели. Используя цикл, мы перебираем все найденные b-box, и выводим в консоль результаты.

```
(venv) M:\FromNotebook\Main\DumpToLaptop\Python Scripts\PracticeMuiv\Этапы\2>python main.py
image 1/1 M:\FromNotebook\Main\DumpToLaptop\Python Scripts\PracticeMuiv\2\image.jpg: 448x640 1 bottle, 2 bananas, 1 apple,
2 oranges, 1 dining table, 1 vase, 109.1ms
Speed: 2.8ms preprocess, 109.1ms inference, 3.0ms postprocess per image at shape (1, 3, 448, 640)
orange 0.7963771224021912 [425.1127014160156, 838.3732299804688, 829.4659423828125, 1169.5908203125]
orange 0.7457260489463806 [570.8458251953125, 559.57275390625, 936.89794921875, 1110.7628173828125]
apple 0.6648644208908081 [127.77294921875, 412.3387756347656, 641.8629150390625, 961.8473510742188]
vase 0.627534806728363 [922.2898559570312, 71.16044616699219, 1517.193603515625, 1250.619384765625]
dining table 0.46103623509407043 [75.5015640258789, 744.2622680664062, 2046.2135009765625, 1345.5775146484375]
banana 0.4317740499973297 [91.41670227050781, 922.2310791015625, 1013.3511962890625, 1340.139892578125]
banana 0.3619215786457062 [89.75834655761719, 929.3753051757812, 719.4384765625, 1336.42626953125]
bottle 0.33642593026161194 [104.424072265625, 3.4441895484924316, 649.9828491210938, 671.4141235351562]
```

Рисунок 4 - Анализ выходных данных

Рисунок 4 демонстрирует результаты, полученные при обработке входного изображения, показанного на рисунке 2. По проведенному анализу можно понять, что выходные данные модели имеют удобную структуру и благодаря чему можно их выводить в удобном для чтения вида, чтобы дальше с ними работать.

5.3 Обработка видео

Следующим шагом стала обработка видео с покадровым анализом. Для этого нам понадобится ранее установленная библиотека OpenCV. Она имеет инструменты для чтения и записи видеопотока. Чтобы правильно обработать видео мы должны нарезать его на кадры и каждый кадр прогнать через YOLO, модель выполняет обнаружение объектов и отображает b-box, класс и насколько она уверена. После чего все эти кадры снова связать в одно видео.



Рисунок 5 - Один из кадров входного видео

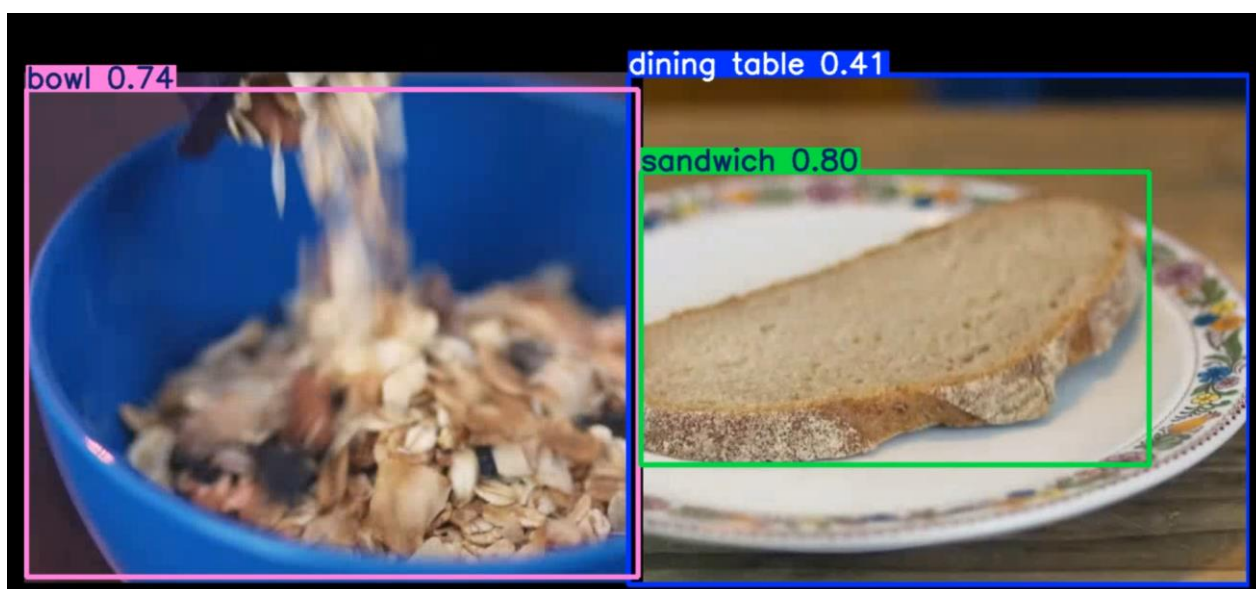


Рисунок 6 - Результат

Благодаря обработке видеопотока мы смогли убедиться, что модель может отлично справляться с динамическими сценами, что открывает перспективы ее применения для анализа видео, например в реальном времени.

5.4 Фильтрация объектов по классам

Так как модель YOLO умеет распознавать сразу множество разных классов, иногда полезно обнаруживать только определенные категории.



Рисунок 7- Исходное изображение

Возьмем в пример рисунок 7, на нем мы видим группу людей, а также слева заметна машина. Я поставил перед собой задачу отметить только объекты с классом “person”, то есть людей. Для решения этой задачи в библиотеке ultralytics можно в модель передать аргумент classes с ID нужных классов. В нашем случае нам нужен класс person с ID равным 0. Чтобы не запоминать каждый раз ID для каждого класса, я решил добавить в код фильтр, который по названию класса находит его ID.



Рисунок 8 - Результат

Как мы видим на рисунке 8 нашлись только люди, для машины b-box не был размечен. Данный функционал можно использовать, если на фото, либо видео множество объектов, а нам они все не нужны. Например, если перед нами стоит задача считать покупателей в магазине или автомобилей на парковках.

5.5 Подсчет объектов в реальном времени

Следующим этапом передо мной встала задача считать в реальном времени с использованием веб-камеры, найденные моделью объекты. Для решения этой задачи опять же использовалась библиотека OpenCV, а именно функция захвата видеопотока с камеры. Каждый полученный кадр передается YOLOv8, после чего идет подсчет всех найденных объектов по классам, а точнее по ID классам, и всю полученную статистику отображает на видеопотоке.



Рисунок 9 - Результат

Как видно на рисунке 9, счетчик корректно считает найденные объекты и также работает на нескольких разных классах. Подобные системы можно

использовать в системах безопасности, при подсчете посетителей и мониторинге рабочих зон. После проведения исследования могу с уверенностью утверждать, что данная модель за счет своей высокой скорости можно использовать при анализе данных с камер реального времени.

5.6 Реализация зоны интереса (ROI)

ROI – это заранее определенная область на кадре, только внутри которого происходит анализ объектов. Перед началом работы алгоритма необходимо задать эту область. Как сказано в условии задачи в качестве центра взял формулу $((x_min + x_max) / 2, (y_min + y_max) / 2)$. Алгоритм должен проверять попадания объектов внутрь ROI и учитывать только при выполнении условия. Попадание проверяется условием $x_roi_min \leq center_x \leq x_roi_max$ для x и $y_roi_min \leq center_y \leq y_roi_max$ для y .

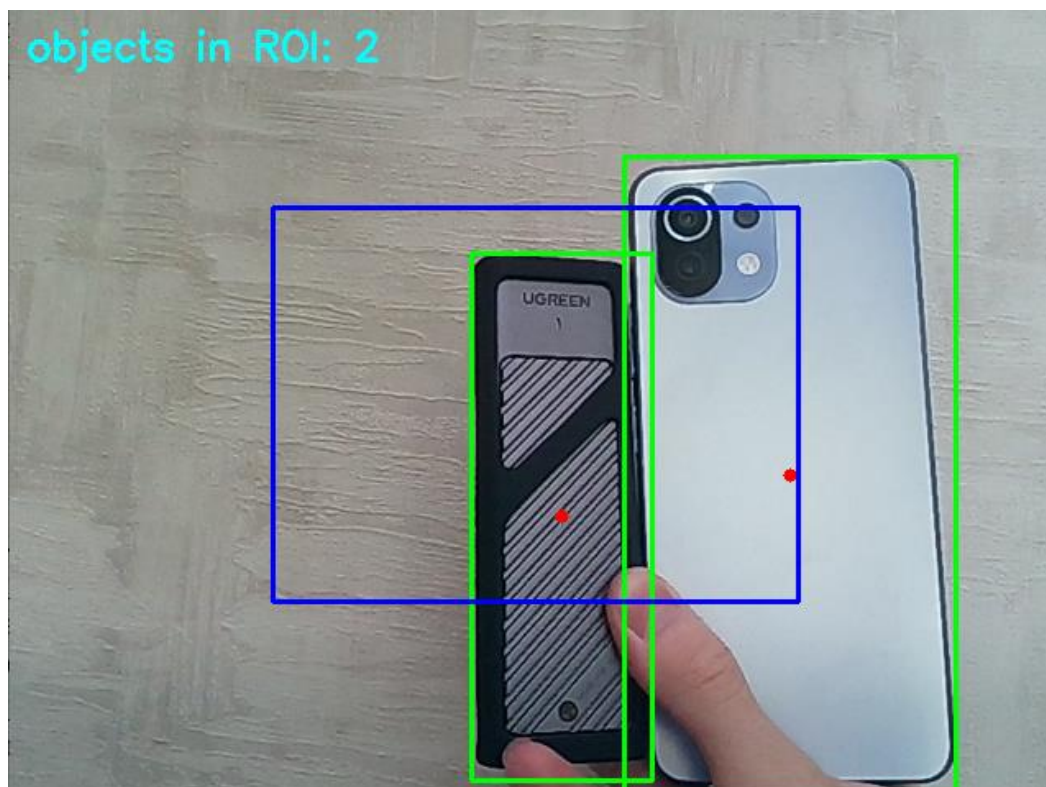


Рисунок 10 - Результат с двумя объектами ROI

На рисунке 10 синим прямоугольником отмечена зона ROI, зелеными найденные объекты с красным центром внутри. Если у детектированного объекта центр выходит за рамку ROI, он должен пропадать из счетчика

объектов. Вот, например на рисунке 11, видно, что центр телефона вышел за рамки ROI, и поэтому он не отмечается b-box и не учитывается в счетчике.

Этот алгоритм очень часто используют в реальных системах.

Например, при помощи него считают покупателей возле кассы, чтобы не анализировать всю картинку с видеокамеры, берут область около кассы и на вход модели идет только она. Это позволяет не тратить мощности понапрасну и исключить нерелевантные объекты. Или например на входе в кинотеатр, чтобы не считать людей которые проходят мимо кинотеатра.

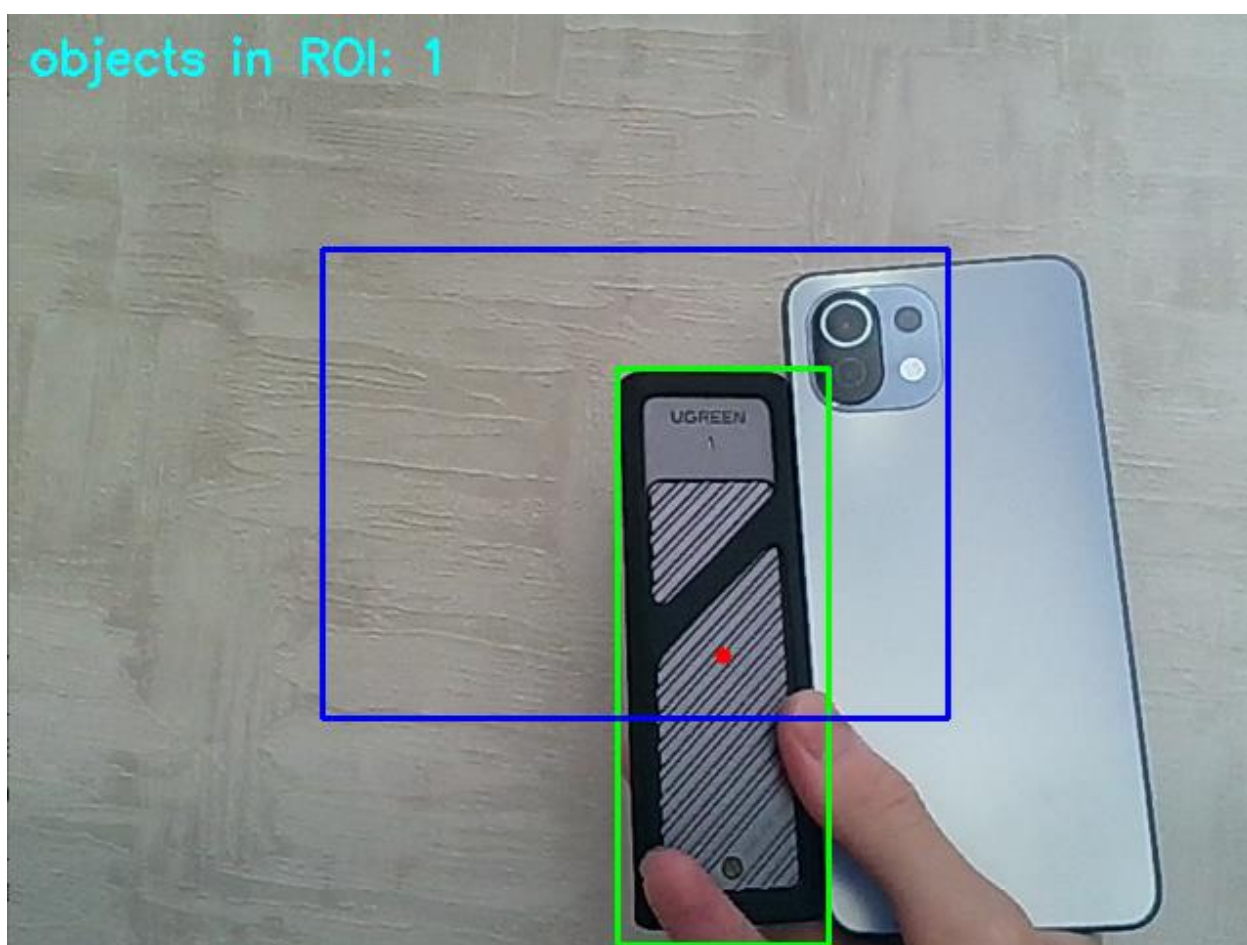


Рисунок 11 - Результат с одним объектом ROI

5.7 Исследование порога уверенности

В рамках этого исследования было изучено, как изменится точность распознавания объектов, если поменять параметр “confidence threshold”.

Уровень confidence характеризует степень уверенности модели в

правильности идентифицированного объекта. Мною были протестированы значения параметра 0.1, 0.3, 0.5 и 0.7.

```
(venv) M:\FromNotebook\Main\DumpToLaptop\Python Scripts\PracticeMuiv
\Этапы\7>python main.py
conf=0.1: detections=8
conf=0.3: detections=4
conf=0.5: detections=2
conf=0.7: detections=1
```

Рисунок 12- Результаты

Корректировка параметра может помочь уменьшить количество ложноположительных программ.



Рисунок 13- Результат с conf=0.1



Рисунок 14 - Результат с conf=0.7

Как мы видим по рисункам 13 и 14 чем ниже порог, тем сильнее увеличивается количество обнаружений, но одновременно и с этим возрастает число ошибок. При высоком пороге ошибки уменьшаются, но часть реально возможных детекций пропускается.

5.8 Изучение метрик качества

Самым последним этапом работы стало определение качества детектирования объектов с помощью метрик. Был взят датасет изображений с ручной разметкой объектов в формате PascalVOC XML. Мною был проведен анализ, где я сопоставлял координаты реальных объектов с теми, что предсказала модель YOLOv8, после чего сравнивал насколько она ошиблась. После того, как я запустил скрипт расчета метрик по моему датасету, мне выдало результат с рисунка 15.



```
TP: 40
FP: 30
FN: 12
Precision: 0.571
Recall: 0.769
F1: 0.656
```

Рисунок 15 - Метрики

Проанализировав их, мы можем сделать вывод на основе Precision, что 57.1% от всех положительных объектов были определены верно. Recall указывает, на то что модель обнаружила 76.9% от всех реально существующих объектов.

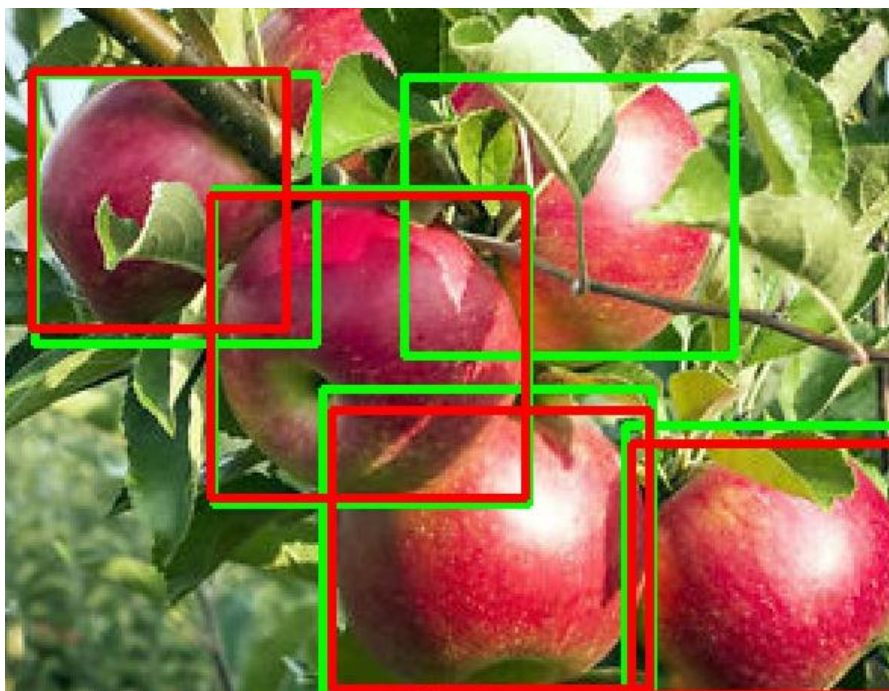


Рисунок 16 - Предсказания моделей

На рисунке 16 вы можете увидеть зеленые и красные рамки. Зеленые относятся к правильным ответам, а красные те, что предсказала YOLOv8. В конечном итоге после всего анализа можно говорить, что модель успешно обнаруживает большинство реальных объектов. Комбинированный показатель F1-score показывает среднюю эффективность модели. Чтобы улучшить точность модели можно повысить порог уверенности(conf).

6.Вывод

В рамках практической работы мы успешно освоили передовую технологию обнаружения объектов с помощью нейросети YOLOv8. В процессе исследования были освоены ключевые аспекты применения предобученных моделей компьютерного зрения, что позволяет избежать необходимости самостоятельного обучения нейронных сетей и соответствует современным методологическим подходам в разработке интеллектуальных систем. Оценка эффективности модели YOLOv8 осуществлялась посредством стандартных метрик, таких как IoU, Precision, Recall и F1-score, что является общепринятой практикой в задачах детекции объектов. Таким образом, поставленная цель практической работы была достигнута в полном

объеме. Были сформированы практические компетенции в области работы с современными инструментами компьютерного зрения, изучены принципы функционирования моделей детектирования объектов, а также осуществлена разработка прикладной системы анализа изображений и видеопотока на основе YOLOv8. Полученные знания и навыки могут быть применены при создании систем интеллектуального видеонаблюдения, автоматизированного контроля объектов, анализа поведенческих характеристик и других прикладных решений в области искусственного интеллекта.

Список используемой литературы

1. Solem, J. E. Programming Computer Vision with Python: Tools and algorithms for analyzing images / J. E. Solem. – 1st. – Sebastopol, CA : O'Reilly Media, 2012. – 260 с. – ISBN 978-1449316549.
2. Wikipedia : сайт. – URL:
[https://en.wikipedia.org/wiki/Neural_network_\(machine_learning\)](https://en.wikipedia.org/wiki/Neural_network_(machine_learning)) (дата обращения: 15.05.2026)
3. Neural networks and deep learning : сайт. – URL:
<http://neuralnetworksanddeeplearning.com/about.html> (дата обращения: 15.05.2026)
4. Neurons in Deep Learning—The Building Blocks of Neural Networks // Coursera : сайт. – URL: <https://www.coursera.org/articles/neurons-in-deep-learning> (дата обращения: 16.05.2026)
5. Howse, J. OpenCV: Computer Vision Projects with Python / J. Howse, P. Joshi, M. Beyeler. – Birmingham B3 2PB, UK. : Packt Publishing Ltd., 2016. – 558 с. – ISBN 978-1-78712-549-0.